P2845R8
Formatting of std::filesystem::path

Published Proposal, 2024-03-19

Author:
Victor Zverovich

Audience:
SG16, LEWG

Project:
ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Table of Contents

1 Introduction

2 Changes from R7

3 Changes from R6

4 Changes from R5

5 Changes from R4

6 Changes from R3

7 Changes from R2

8 Changes from R1

9 Changes from R0

10 SG16 Poll Results (R6)

11 LEWG Poll Results (R4)

12 SG16 Poll Results (R2)

13 Problems

14 Proposal

15 Wording

16 Implementation

17 Acknowledgements

References

Informative References

"The Tao is constantly moving, the path is always changing." — Lao Tzu

§ 1. Introduction

[P1636] "Formatters for library types" proposed adding a number of `std::formatter` specializations, including the one for `std::filesystem::path`. However, SG16 recommended removing it because of quoting and localization concerns. The current paper addresses these concerns and proposes adding an improved `std::formatter` specialization for `path`.

§ 2. Changes from R7

- Changed `format_parse_context` to `basic_format_parse_context<charT>` in the wording.
- Replaced "`formatter<filesystem::path, charT>` is *debug-enabled*" with defining the `set_debug_format` function explicitly.
- Replaced `Char` with `charT` in the wording.
- Replaced `p.generic()` with `p.generic_string<filesystem::path::value_type>()` in the wording.
- Added a feature testing macro.

§ 3. Changes from R6

- Added SG16 poll results for R6.

§ 4. Changes from R5

- Added generic format support per LWG feedback.

§ 5. Changes from R4

- Replaced "invalid code units" with a more specific "maximal subparts of ill-formed subsequences" per LEWG feedback.
- Added LEWG poll results for R4.

§ 6. Changes from R3

- Added SG16 poll results.

§ 7. Changes from R2

- Added missing `:?` to the escaping example in Proposal.
- Changed the wording around the escaping example to not mention hexadecimal escapes since Unicode escapes may be produced as well.

§ 8. Changes from R1

- Provided control over escaping via format specifiers per SG16 feedback.

§ 9. Changes from R0

- Added a reference to [format.string] for the productions *fill-and-align* and *width*.
- Replaced *range-format-spec* with *path-format-spec* in the *Effects* clause of the `format` function.
- Added missing transcoding to the definition of the `format` function.

§ 10. SG16 Poll Results (R6)

POLL: Forward P2845R6 to LEWG.

Outcome: Unanimous consent to forward.

§ 11. LEWG Poll Results (R4)

POLL: Forward P2845R4 (Formatting of std::filesystem::path) with modified wording for Effects to use the term "replacement of a maximal subpart" to LWG for C++26 to be confirmed with a Library Evolution electronic poll.

SF	F	N	A	SA

11	9	0	0	0

Outcome: Unanimous consent to forward.

§ 12. SG16 Poll Results (R2)

POLL: Forward P2845R2, Formatting of `std::filesystem::path`, to LEWG with a recommended target of C++26.

SF	F	N	A	SA
5	2	1	0	0

Outcome: Strong consensus.

(The poll states P2845R2, but the revision of the paper that was reviewed was a draft of P2845R3 that addressed some minor issues.)

§ 13. Problems

[P1636] proposed defining a `formatter` specialization for `path` in terms of the `ostream` insertion operator which, in turn, formats the native representation wrapped in `quoted`. For example:

```
std::cout << std::format("{}", std::filesystem::path("/usr/bin"));
```

would print `"/usr/bin"` with quotes being part of the output.

Unfortunately this has a number of problems, some of them raised in the LWG discussion of the paper.

First, `std::quoted` only escapes the delimiter (`"`) and the escape character itself (`\`). As a result the output may not be usable if the path contains control characters such as newlines. For example:

```
std::cout << std::format("{}", std::filesystem::path("multi\nline"));
```

would print

```
"multi
line"
```

which is not a valid string in C++ and many other languages, most importantly including shell languages. Such output is pretty much unusable and interferes with formatting of ranges of paths.

Another problem is encoding. The `native` member function returns `basic_string<value_type>` where

> `value_type` is a `typedef` for the operating system dependent encoded character type used to represent pathnames.

`value_type` is normally `char` on POSIX and `wchar_t` on Windows.

This function may perform encoding conversion per [fs.path.type.cvt].

On POSIX, when the target code unit type is `char` no conversion is normally performed:

> For POSIX-based operating systems `path::value_type` is `char` so no conversion from `char` value type arguments or to `char` value type return values is performed.

This usually gives the desired result.

On Windows, when the target code unit type is `char` the encoding conversion would result in invalid output. For example, trying to print the following path in Belarusian

```
std::print("{}\n", std::filesystem::path(L"Шчучыншчына"));
```

would result in the following output in the Windows console even though all code pages and localization settings are set to Belarusian and both the source and literal encodings are UTF-8:

```
"??????????"
```

The problem is that despite `print` and `path` both support Unicode the intermediate conversion goes through CP1251 (the code page used for Belarusian) which is not even valid for printing in the console which uses legacy CP866. This has been discussed at length in [P2093] "Formatted output".

§ 14. Proposal

Both of the problems discussed in the previoius section have already been solved. The escaping mechanism that can handle invalid code units has been introduced in [P2286] "Formatting Ranges" and encoding issues have been addressed in [P2093] and other papers. We apply those solutions to the formatting of paths.

This paper proposes adding a `formatter` specialization for `path` that does escaping similarly to [P2286] and Unicode transcoding on Windows. Additionally, it proposes giving the user control over escaping via format specifiers. The debug format (`?`) gives the escaped representation while the default is unescaped and minimally processed with only invalid code units substituted with replacement characters if necessary. This is consistent with formatting of strings. The default format can be useful for displaying paths in a UI and gives the user control whether and how to handle special characters. The debug format is useful for displaying paths as parts of a larger structure such as a range and prevents interferring with its formatting.

Code	P1636	This proposal
`auto p = std::filesystem::path("/usr/bin");` `std::cout << std::format("{}", p);`	`"/usr/bin"`	`/usr/bin`
`auto p = std::filesystem::path("multi\nline");` `std::cout << std::format("{}", p);`	`"multi` `line"`	`multi` `line`
`auto p = std::filesystem::path("multi\nline");` `std::cout << std::format("{:?}", p);`	ill-formed	`"multi\nline"`

`// On Windows with UTF-8 as a literal encoding.`
`auto p = std::filesystem::path(L"Шчучыншчына");`
`std::print("{}\n", p);`
	`"►►►►►►►►►►►►"`	`Шчучыншчына`

This leaves only one question of how to handle invalid Unicode. Plain strings handle them by formatting ill-formed code units as hexadecimal escapes, e.g.

```
// invalid UTF-8, s has value: ["\x{c3}("]
std::string s = std::format("[{:?}]", "\xc3\x28");
```

This is useful because it doesn't loose any information. But in case of paths it is a bit more complicated because the string is in a different form and the mapping between ill-formed code units in one form to another may not be well-defined.

When escaping, the current paper proposes applying it to the original ill-formed data because it gives more intuitive result and doesn't require non-standard mappings such as WTF-8 ([WTF]).

For example:

```
auto p = std::filesystem::path(L"\xd800"); // a lone surrogate
std::print("{:?}\n", p);
```

prints

```
"\u{d800}"
```

When not escaping, the paper proposes substituting invalid code units with replacement characters which is the recommended Unicode practice ([UNICODE-SUB]):

For example:

```
auto p = std::filesystem::path(L"\xd800"); // a lone surrogate
std::print("{}\n", p);
```

prints

```
<U+FFFD>
```

§ 15. Wording

Add an entry for `__cpp_lib_format_path` to section "Header `<version>` synopsis" [version.syn], in a place that respects the table's current alphabetic order:

```
#define __cpp_lib_format_path **placeholder**  // also in <filesystem>
```

Add to "Header <filesystem> synopsis" [fs.filesystem.syn]:

```
// [fs.path.fmt], formatter
template<class charT> struct formatter<filesystem::path, charT>;
```

Add a new section "Formatting" [fs.path.fmt] under "Class path" [fs.class.path]:

```
template<class charT> struct formatter<filesystem::path, charT> {
  constexpr void set_debug_format();

  constexpr typename basic_format_parse_context<charT>::iterator
    parse(basic_format_parse_context<charT>& ctx);

  template<class FormatContext>
    typename FormatContext::iterator
      format(const filesystem::path& path, FormatContext& ctx) const;
};
```

```
constexpr void set_debug_format();
```

Effects: Modifies the state of the `formatter` to be as if the *path-format-spec* parsed by the last call to `parse` contained the `?` option.

```
constexpr typename basic_format_parse_context<charT>::iterator
  parse(basic_format_parse_context<charT>& ctx);
```

Effects: Parses the format specifier as a *path-format-spec* and stores the parsed specifiers in `*this`.

path-format-spec:
fill-and-align$_{opt}$ *width*$_{opt}$ `?`$_{opt}$ `g`$_{opt}$

where the productions *fill-and-align* and *width* are described in [format.string]. If the `?` option is used then the path is formatted as an escaped string ([format.string.escaped]).

Returns: An iterator past the end of the *path-format-spec*.

```
template<class FormatContext>
  typename FormatContext::iterator
    format(const filesystem::path& p, FormatContext& ctx) const;
```

Effects: Let `s` be `p.generic_string<filesystem::path::value_type>()` if the `g` option is used, otherwise `p.native()`. Writes `s` into `ctx.out()`, adjusted according to the *path-format-spec*. If `charT` is `char`, `path::value_type` is `wchar_t` and the literal encoding is UTF-8 then the escaped path is transcoded from the native encoding for wide character strings to UTF-8 with maximal subparts of ill-formed subsequences substituted with U+FFFD REPLACEMENT CHARACTER per the Unicode Standard, Chapter 3.9 U+FFFD Substitution in Conversion. If `charT` and `path::value_type` are the same then no transcoding is performed. Otherwise, transcoding is implementation-defined.

Returns: An iterator past the end of the output range.

§ 16. Implementation

The proposed `formatter` for `std::filesystem::path` has been implemented in the open-source {fmt} library ([FMT]).

§ 17. Acknowledgements

Thanks to Mark de Wever, Roger Orr and Tom Honermann for reviewing an early version of the paper and suggesting a number of fixes and improvements. Thanks Jonathan Wakely for wording suggestions.

§ References

§ Informative References

[FMT]
Victor Zverovich; et al. *The {fmt} library*. URL: https://github.com/fmtlib/fmt

[P1636]

Lars Gullik Bjønnes. *Formatters for library types*. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1636r2.pdf

[P2093]

Victor Zverovich. *Formatted output*. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p2093r14.html

[P2286]

Barry Revzin. *Formatting Ranges*. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p2286r8.html

[UNICODE-SUB]

The Unicode Consortium. *The Unicode Standard Version 13.0 – Core Specification, Chapter 3.9, U+FFFD Substitution of Maximal Subparts*. URL: https://www.unicode.org/versions/Unicode13.0.0/UnicodeStandard-13.0.pdf

[WTF]

Simon Sapin. *The WTF-8 encoding*. URL: https://simonsapin.github.io/wtf-8/